TITLE OF THE THESIS

SUBTITLE OF THE THESIS

Gustavo Luvizotto Cesar

Graduation Committee

Chairman:	TBD
Supervisor:	TBD
Supervisor:	TBD
Co-supervisor:	TBD

Members:

prof.dr.	TBD	University of Twente, The Netherlands
dr.ir.	TBD	University of Twente, The Netherlands
prof.dr.ir.	TBD	TBD
prof.dr.-ing.	TBD	TBD

Funding sources
TBD

UNIVERSITY OF TWENTE. | DIGITAL SOCIETY INSTITUTE

DSI Ph.D. Thesis Series No. TBD
Digital Society Institute
P.O. Box 217, 7500 AE
Enschede, The Netherlands

ISBN: TBD
ISSN: TBD (DSI Ph.D. Thesis Series No. TBD)
DOI: TBD
TBD

Typeset with LaTeX. Printed by Ipskamp Printing.
Cover design by TBD.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
http://creativecommons.org/licenses/by-nc-sa/4.0/

TITLE OF THE THESIS

SUBTITLE OF THE THESIS

DISSERTATION

to obtain
the degree of doctor at the University of Twente,
on the authority of the rector magnificus,
prof.dr.ir. A. Veldkamp,
on account of the decision of the Doctorate Board,
to be publicly defended
on TBD the TBDth of TBD 20XX at TBD

by

Gustavo Luvizotto Cesar

born on January 1, 1970
in the Netherlands

This dissertation has been approved by:

Supervisors:
TBD
TBD

Co-supervisor:
TBD

Acknowledgements

That's all folks.

Abstract

Test

Samenvatting

Nederlands taal.

Contents

1 Introduction **1**

2 Background **3**
2.1 Anycast 4
2.2 TLS 4
2.3 LDAP 9
2.4 Email Security Protocols 11
2.5 Contributions 13

3 Sample Chapter **15**

4 Conclusions **17**

Bibliography **19**

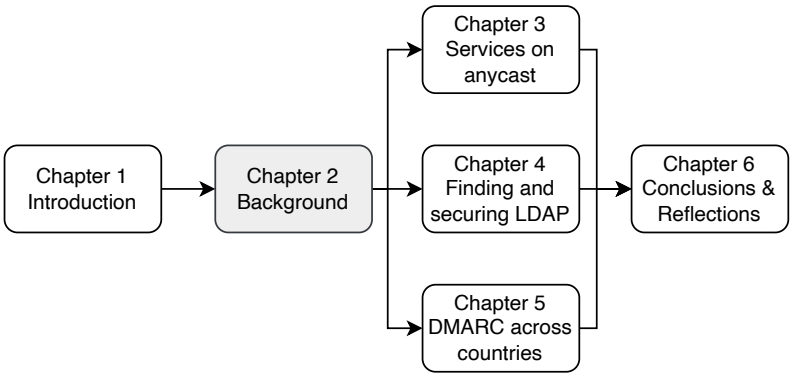
CHAPTER 1

Introduction

My intro [1].

CHAPTER 2

Background



This chapter provides background information on Anycast, Transport Layer Security (TLS), Lightweight Directory Access Protocol (LDAP), and email security protocols such as Sender Policy Framework (SPF), DomainKeys Identified Mail (DKIM), and Domain-based Message Authentication, Reporting and Conformance (DMARC). This information is necessary to to understand the subsequent chapters. Our goal is not to provide an exhaustive description of the aforemetioned topics, but rather offer sufficient details relevant to this thesis.

2.1 Anycast

The first RFC describing anycast (RFC 1546 [2]) was published in 1993 and explains how anycast simplifies finding an appropriate server. As service examples, they list DNS, FTP, and "Archie" (one of the first Internet search engines). Nowadays, anycast is widely deployed for critical Internet infrastructure, such as the DNS [3], or by Content Delivery Networks (CDNs) to provide web services [4, 5].

Anycast is the practice of making a service IP address (either IPv4 or IPv6) available in multiple physical locations [6]. Such locations are referred to catchments or simply as Points of Presence (PoPs). For instance, when a user is located in Enschede trying to access a website that is anycasted in multiple locations such as Amsterdam, Rio de Janeiro and Sydney, the user's request will be sent to Amsterdam. The routing decision is made by Border Gateway Protocol (BGP) which directs the request to a single PoP, typically one that is topologically close to the user [6] from a routing, and often geographically [7], perspective.

A common misconception is that announcing an address at multiple locations, in itself, is classified as anycast. Many CDNs announce their prefixes at multiple network edges (in different geographic regions), where they route traffic internally to a single PoP hosting the service. Critically, as defined in RFC 4786 [6], anycast is the replication of a service address where several PoPs make the service available.

There are multiple reasons for providing a service using anycast. Anycast can increase service resilience through automatic failover. For instance, in our previous example, if the Amsterdam PoP is unavailable, the user's request would be redirected to the next best PoP (either Rio de Janeiro or Sydney, depending on routing decisions). Furthermore, anycast offers service replication closer to users to reduce latency, and enables load distribution across PoPs.

2.2 TLS

The Transport Layer Security (TLS) is a protocol that provides secure communications over the network via a set of cryptographic algorithms and certificate-based authentication. Historically, TLS was introduced in 1994 with Secure Socket Layer (SSL) version 1.0 [8] and Internet Engineering Task Force (IETF) standardized TLS in 1999 and the current version is 1.3 [9], while 1.2 [10] is still widely used and not considered deprecated. Examples of known application-layer protocols that use TLS are HTTPS to communicate with web servers, SMTPS for email servers and LDAPS for directory services. Two parties communicate using TLS after a successful handshake.

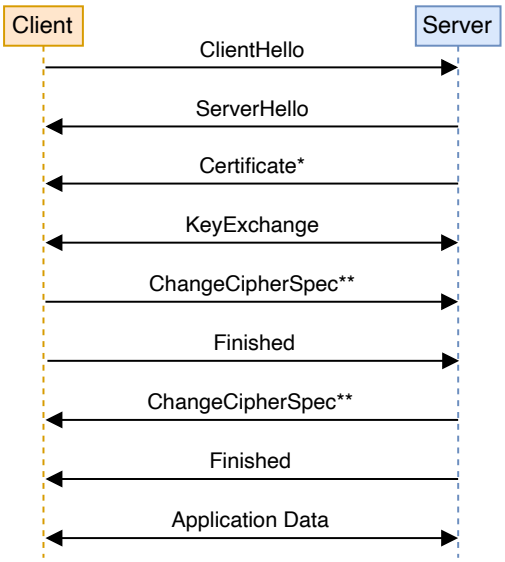


Figure 2.1: Simplified view of the TLS handshake. * indicates that the message is optional

2.2.1 TLS handshake

The TLS handshake occurs right after a Transmission Control Protocol (TCP) connection is established between a client and a server. We depict a simplified view of the TLS handshake in Fig. 2.1, where we explain the key steps and ideas. We refer to the literature for other messages and details necessary for implementation of the protocol [10, 9].

The TLS handshake starts with the client sending a `ClientHello` message. This message contains the client's highest supported TLS version, session ID, a list of supported cipher suites (from the strongest to the weakest), compression methods (used to compress data prior to encryption) it supports and a random value. The session ID is optional and when present, allows the server to resume a previous session reusing TLS configurations and keys from a server-cached handshake and the handshake proceeds directly to the `ChangeCipherSpec` and `Finished` messages. Otherwise, a new session ID is generated by the server and sent in the `ServerHello` message. The client random value is used to generate

the master secret alongside the server's random value from the `ServerHello` message.

The server replies with a `ServerHello` message that contains the selected TLS version, a random value, session ID, selected cipher suite, the selected compression method and optionally extensions (*e.g.*, the Server Name Indication (SNI) [11]). The server chooses the best (most secure) TLS version and cipher suite which are supported by both parties. The selected cipher suite determines the cryptographic primitives used in subsequent handshake steps, including the key exchange mechanism, authentication method, symmetric encryption algorithm, and message authentication algorithm. In case the parties do not agree on a common TLS version or cipher suite, the handshake fails, alert messages are exchanged between the client and server and the connection is closed. We provide more details about TLS cipher suites in Section 2.2.2.

The server sends its X.509 certificate first (leaf or end-host certificate) and a chain of certificates in the `Certificate` message. The distribution of certificates, its content and its validation is briefly discussed in Section 2.2.3. The client uses the certificates to authenticate the server. For instance, by validating the chain, and the validity period of the certificates, and most importantly, by verifying the signature of the leaf certificate using the public key of the server. The signature is computed over a hash of the certificate contents using the issuer's private key. The client can compute the digest and verify against the decrypted signature using the server's public key. This process ensures that the certificate is not tampered and that the client is communicating with the intended server. The authentication algorithm specified in the cipher suite (*e.g.*, Rivest-Shamir-Adleman (RSA) or Elliptic Curve Digital Signature Algorithm (ECDSA)) determines how the certificate's public key is used to verify the server's identity.

Client and server perform the key exchange process. The key exchange mechanism is determined by the selected cipher suite (for TLS $\leq$1.2), *e.g.*, RSA, Diffie-Hellman (DH), or Elliptic Curve Diffie-Hellman (ECDH). This choice defines how the premaster secret is established during the handshake. The key exchange allows client and server to generate a premaster secret that is used to derive the session keys. The premaster secret is transmitted either by encrypting using the server's public key (in RSA) or by using the DH key exchange method to generate the secret without transmitting it. The public parameters of the DH can be statically defined in the certificate [12] or generated on every session, which is called ephemeral. Ephemeral key exchange methods provide Perfect Forward Secrecy (PFS), which means that even if the server's private key is compromised, past communications remain secret. The derived session keys are used to encrypt and decrypt the application data during the TLS session. The encryption algorithm specified in the cipher suite (*e.g.*, Advanced Encryption Standard (AES) with Cipher Block Chaining (CBC)

or Galois Counter Mode (GCM)) are only applied after the handshake keys have been established.

Both client and server send a `ChangeCipherSpec` message to indicate that subsequent messages will be encrypted using the negotiated cipher suite and session keys. Finally, both parties exchange a `Finished` message to verify that the handshake was successful. The `Finished` message contains a hash of all handshake messages, which is encrypted using the session key. The hash algorithm (*e.g.*, SHA256) associated with the cipher suite is used and this approach ensures the integrity of the handshake.

The modern TLS version 1.3 handshake simplifies the negotiation process. For instance, the `ClientHello` message includes supported cipher suites, key shares for ephemeral key exchange, and optionally Pre-Shared Keys (PSKs). The server responds with a `ServerHello` message selecting the cipher suite and key share after which parties derive the session keys, send the certificates, the PSK and early data. After that, the server immediately sends the `Finished` message, and the client computes the verifications and sends its `Finished` message. In TLS 1.3, only ephemeral key exchange methods are used, providing forward secrecy by default. Furthermore, the TLS v1.3 handshake allows for 0-Round-Trip Time (RTT) handshake, which means that the client can send early application data in the first message to the server, using the computed session key.

2.2.2 Cipher suites

The TLS cipher suites represent a set of cryptographic parameters agreed during the TLS handshake used in a TLS session. Cipher suites of TLS version $\leq$1.2 have similar format as: `TLS_ECDH_RSA_WITH_AES_256_CBC_SHA`. Here, `ECDH` is the key exchange algorithm, `RSA` is the authentication algorithm, `AES_256` is the session cipher and its bit size, `CBC` is the type of encryption, and `SHA` is the digest or message authentication code algorithm.

For TLS v1.3, the cipher suite format is simplified to `TLS_AES_256_GCM_SHA384`, where the key exchange are (EC)DHE, PSK or a combination of both and are ommited from the cipher suite. In TLS 1.3, the separation of key exchange and authentication from cipher suites simplifies negotiation, while also enforcing the use of authenticated encryption schemes.

2.2.3 TLS and the PKI

We briefly describe the structure of X.509 certificates and the public key infrastructure (PKI) used to authenticate servers during the TLS handshake. X.509 certificates follow the hierarchical model defined in X.500 and are standardized by ITU-T [13]. The X.509 certificates were created to authenticate users to read

and write information in directory services [14]. Over time, X.509 certificates have been widely adopted in applications such as S/MIME [15] and services secured with TLS, including Web and LDAP [16], with the Web driving their large-scale deployment [17].

X.509 certificates contain the following fields [13]: the version (typically version 3), the serial number (unique unique per issuing Certificate Authority (CA)), the Subject and Issuer information (*e.g.*, Common Name, Organization name and Organization Unit), the subject public key with the algorithm identifier and the key, the validity period of the certificate (*not before* and *not after*), and the signature algorithm identifier and signature of the certificate authority that issued the certificate. X.509 certificates may include optional extensions, where the most common and relevant to this thesis are: Subject Alternative Name (SAN) and the *CA* flag. For instance, the SAN extension specifies additional identities associated with the certificate, such as domain names or IP addresses. The *CA* flag extension, which when set to "true", indicates that the certificate can be used to sign other certificates. In chapter (TODO ref LDAP) we show that LDAP servers use broken chains of certificates with the *CA* flag set to "true" in the leaf certificates.

X.509 certificates follow a hierarchical structure that enables a chain of trust. For instance, a leaf certificate is usually signed by an intermediate certificate, until reaching a trusted root certificate. A chain can have multiple intermediate certificates in the chain, or the leaf can be directly signed by a root certificate. The intermediate and root certificates are issued by a trusted CA. The CA is responsible for verifying the identity of certificate requesters. Hence, CA are critical entities in the Public Key Infrastructure (PKI) ecosystem., and reaching such level of trust is a complex process from well-audited forums such as the CA/Browser Forum [18]. When a CA goes rogue, it can issue fraudulent certificates. As a consequence, rogue CAs are removed from the trusted root store, such as the case of DigiNotar removed from browsers [19]. Trusted root stores are maintained and embedded in the Operational Systems (OSs) (*e.g.*, Linux, Apple, and Windows), browsers (*e.g.*, Chrome), or maintained by other entities (*e.g.*, Common CA Database (CCADB)[1]). Certificates can also be self-signed, however their use is generally discouraged and limited to specific scenarios (see Section 2.3.3). When certificates are signed by a CA, it creates a cryptographic binding between the identity of parent and child certificates. This binding enables clients to verify the authenticity of certificates during the TLS handshake.

Certificate validity depends on several conditions. For instance, valid certificates have the chain of trust from the leaf certificate cryptographically leading to a publicly and trusted root certificate. Additionally, the validity

[1] `ccadb.org`

period by the time of checking must be within the *not before* and *not after* fields. Furthermore, the signature of the leaf certificate must match the issuer's public key, and the signature algorithm should be secure (*e.g.*, SHA-256) [20]. Hostname verification may additionally be performed, where the domain name of the server must match the Common Name or the SAN field of the certificate. Finally, the revocation status of a certificate can be checked using mechanisms such as Certificate Revocation List (CRL) [21] or the Online Certificate Status Protocol (OCSP) [22].

2.3 LDAP

Similar to X.509 certificates, LDAP is also derived from the X.500 standard. Lightweight Directory Access Protocol (LDAP) is a client-server protocol to facilitate transfer and access of data stored in directory services such as OpenLDAP or Active Directory (AD). We use the generic term *LDAP servers* to refer to all servers that speak the protocol and distinguish by directory service as needed. Connection-less LDAP (CLDAP) runs over UDP and it is restricted to a small set of queries. In this thesis we do not investigate. LDAP typically runs over TCP on port 389 (clear text with optional StartTLS) or 636 (implicit TLS), while ports 3268 (clear text/StartTLS) and 3269 (implicit TLS) are reserved for the Global Catalog (GC) service in Microsoft's Active Directory (AD). The GC is an AD role to manage and access copies of data items across a multitude of LDAP servers [23]. LDAP stores data in a tree-like hierarchy. The special Root DSA-Specific Entry (DSE) data entry stores information about the server's functionality in form of attributes. These attributes with a predefined type and a value. The type determines the syntax, semantics, and characteristics of the value. Attribute types are uniquely identified internationally by their Object Identifier (OID) [24, 25]. These attributes hold a unique significance, as they encompass essential information about the server, such as supported controls, extensions, features, LDAP versions, authentication mechanism such as Simple Authentication and Security Layers (SASL), and more [26], which can help identify the LDAP product.

LDAP may use TLS, where a cipher suite is agreed upon and a chain of X.509 certificates is sent to a client. The RFC-9325 gives best-practice recommendations.

2.3.1 LDAP Authentication and Authorization

Clients authenticate to an LDAP server by attempting a bind operation. A connection between the client and the server is established if the bind is successful [27, 28]. As part of the bind request, the client chooses an authentication

method and, if necessary, supplies authentication credentials. LDAPv3 supports four types of authentication: simple, anonymous, unauthenticated, and SASL authentication [28].

The simple authentication uses an username and a cleartext password. This method requires TLS to encrypt the password. Anonymous authentication uses empty username and password. Furthermore, the unauthenticated authentication uses an username but without a password. It's worth noting that RFC 4513 [28] recommends disabling this method by default for clients and servers. This method can be used with Single Sign-On (SSO) to retrieve the Distinguished Name (DN) from the directory to proceed with the authentication. Finally, SASL supports a wide range of authentication methods, such as username/password, Kerberos, and digital certificates. Anonymous and plaintext SASL authentication is not commonly used in LDAP [29]. While some are notably robust, others have been identified as insecure [30].

2.3.2 LDAP message

LDAP messages are encoded using the ASN.1 BER format [27].

LDAP StartTLS

2.3.3 LDAP PKI

A certificate chain must be anchored at a trusted root, requiring the LDAP client to be configured with the required root certificate(s). The so-called end-host (or leaf) certificate provides a cryptographic binding of an identifier—typically the domain name of the LDAP server—to a corresponding public key. On the Web, public CAs issue certificates under CA/Browser Forum rules (*e.g.*, verification of domain ownership). For LDAP, no comparable body or standardization efforts exist. In many cases (*e.g.*, Linux, Windows), the root store that the LDAP client relies on is the same as for the Web. In refined setups, the LDAP server administrator may supply it directly, even using a private CA to issue their own certificates. In this case, the administrators must also deploy the root certificate to their LDAP clients using tools like AD Certificate Services or scripts. In such a *private PKI*, one should issue intermediate certificates from the root certificate, which in turn are used to issue a leaf certificate. This setup is important as it allows the private key for the root certificate to be kept in a secure location. An alternative, semi-private PKI setup is also possible: an operator could request a sub-CA certificate from a public CA, *i.e.*, an intermediate certificate, from which they then issue their own certificates. This setup must be tightly controlled so leaf certificates are only ever issued for domains under control of the administrator [18].

A discouraged practice is to deploy so-called self-signed certificates, where the key used to sign a certificate is also the certified key. In such a case, the client must be pre-configured to accept only this certificate—or users told to ignore the warnings their LDAP client would show. The setup is only suitable if the LDAP server serves a minimal set of users. It also constitutes a security risk as the private key remains online in memory. As certificates are available for free and can be relatively easily deployed with the ACME protocol, there is little justification to use self-signed certificates any more.

2.4 Email Security Protocols

In this section, we provide background information on email security protocols relevant to this thesis (SPF, DKIM, and DMARC) and how DMARC interact with them during email communications.

2.4.1 DKIM

DomainKeys Identified Mail (DKIM) is an email authentication and integrity verification method that employs public-key cryptography. Domain owners use DKIM to digitally sign emails from their domain. During the Simple Mail Transfer Protocol (SMTP) communication, the sender adds the `DKIM-Signature` header field with DKIM relevant tags. For example, the signing domain ("d" tag), a key selector ("s" tag), and a signature (signed with the sender's private key, "b" tag). The recipient's email server queries the sender's Domain Name System (DNS) server for its TXT record (*e.g.*, *<s>._domainkey.<d>.example.com*) to retrieve the public key, and then uses it to verify the received email signature. The verification results can be either *pass*, *permfail*, or *tempfail*. Multiple signatures can be present in an email, and if at least one is valid, DKIM evaluation succeeds.

2.4.2 SPF

Sender Policy Framework (SPF) allows a domain owner to specify a list of IP addresses that are authorized to send emails on behalf of their domain. The domain owner configures the SPF rules through the DNS TXT record. The recipient's mail server extracts the IP address and the domain name from "MAIL FROM" **or** "HELO/EHLO FROM" identify during the SMTP communication. SPF verification is done through the **check_host()** function [31], which uses the identities and the sender's IP address from the SMTP communication, and retrieves the SPF rules to determine whether the sender's IP address belongs to the list of IP addresses configured. The outcome of the verification can be *none* (no conclusion about authorization), *neutral* (treated as "none"), *pass*,

fail, *softfail* (requires further investigation), *temperror* (transient error during verification), or *permerror* (incorrect SPF records). The recipient's mail server can then decide to accept, reject, or mark the email as spam based on the verification result and their own anti-spoofing policies. DMARC uses results of the **check_host()** function based on "MAIL FROM" identity.

2.4.3 DMARC

Domain-based Message Authentication, Reporting and Conformance (DMARC) data is published in the DNS TXT records of the *_dmarc* subdomain (*e.g.*, *_dmarc.example.com*). The DMARC records can be as follows:

```
v=DMARC1; p=reject; rua=mailto:rua@example.com
```

The record always starts with the tag `v=DMARC1` followed by a list of tags separated by semicolons. The tags relevant this thesis are:

- p Policy for organizational domain. It tells the receiving mail server how to handle emails that fail DMARC verification. The possible values are `none` (monitoring only), `quarantine`, and `reject` (block delivery).
- sp Similar to `p`, but for subdomains. If `sp` is not specified, the value of `p` tag is used. If the subdomain has its own `p` tag, then it takes precedence over the `sp` tag.
- rua Specify the email address(es) to which "aggregate reports" (see below) should be sent with "mailto:address". Multiple addresses can be specified.
- ruf Similar to `rua`, but for "forensic reports" (see below).

DMARC uses verification results from SPF and DKIM and also domain name alignment to determine the authenticity of an email. The SPF domain name alignment occurs when the "FROM" domain name matches the "MAIL FROM" domain name of the received email. The DKIM alignment occurs when the domain name present in the "d" tag matches the "MAIL FROM" domain name. DMARC verification passes if SPF check passes and the SPF domain name is aligned **or** DKIM check passes and the DKIM domain name is aligned.

DMARC sends two kinds of reports: **Aggregate** (in `rua` tag) and **Forensic** (in `ruf` tag) [32]. Aggregate reports provide a summary, in XML format, of DMARC, SPF, and DKIM results (pass and fail), the sending IP address, *RFC5322.From* address, etc. The use of forensic reports is limited due to privacy concerns [33, 34]. Reports help domain owners understand their domain's email usage, such as tracking policy effectiveness and spotting abuse or misconfigurations. The nature of forensic reports can lead to parties that

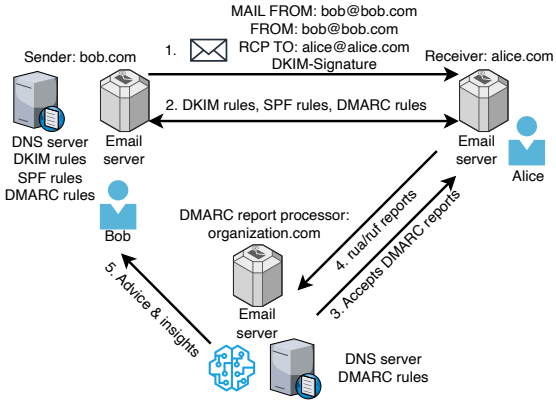


Figure 2.2: Overview of DMARC role in email communication.

receive such reports to have access to sensitive information, although previous work has shown that such reports are falling short of use [33]. DMARC reports are often sent to another domain (*e.g.*, a specialized organization) for processing, as handling raw reports is complex [35].

2.4.4 DMARC role in email communications

Fig. 2.2 depicts a simplified overview of the DMARC role in email communication. Bob (sending domain) sends an email to Alice (receiving domain). Alice queries Bob's DNS server to retrieve DKIM, SPF, and DMARC records to verify the email's authenticity and decide whether to deliver it. When a `rua` address is present, Alice's email server is requested to report to Bob. The reports provide insights into how Bob's domain is used and whether it is legitimate.

2.5 Contributions

This chapter is either extended or adapted version of the author's publications. The Section 2.1 is an adapted version of [36]. The Section 2.2 is new and provides an overview of TLS, yet with information relevant about the TLS handshake, the cipher suites and the PKI that supports TLS. The Section 2.3 is an adaptation of the background sections of [1, 16], and we extend with information about the LDAP message format and the LDAP message to initiate TLS (StartTLS). The Section 2.4 is an extended version of [37] where we provide

extra information about SPF and the alignment process in SPF and DKIM that is used in the process of DMARC verification.

CHAPTER 3

Sample Chapter

Sample chapter [1].

CHAPTER 4

Conclusions

Some conclusions.

Bibliography

[1] J. Kaspereit, G. Öndarö, G. L. Cesar, S. Ebbers, F. Ising, C. Saatjohann, M. Jonker, R. Holz, and S. Schinzel, "LanDscAPe: Exploring LDAP weaknesses and data leaks at internet scale," in *USENIX*, Aug. 2024.

[2] T. Mendez, W. Milliken, and D. C. Partridge, "Host Anycasting Service," Internet Engineering Task Force, RFC 1546, Nov. 1993. [Online]. Available: https://www.rfc-editor.org/info/rfc1546

[3] R. Sommese, G. Akiwate, M. Jonker, G. C. Moura, M. Davids, R. v. Rijswijk-Deij, G. M. Voelker, S. Savage, A. Sperotto *et al.*, "Characterization of anycast adoption in the dns authoritative infrastructure," in *TMA*. ACM, 2021.

[4] D. Cicalese, J. Augé, D. Joumblatt, T. Friedman, and D. Rossi, "Characterizing ipv4 anycast adoption and deployment," in *Proc. CoNEXT*. New York, NY, USA: ACM, 2015. [Online]. Available: https://doi.org/10.1145/2716281.2836101

[5] D. Giordano, D. Cicalese, A. Finamore, M. Mellia, M. Munafò, D. Z. Joumblatt, and D. Rossi, "A First Characterization of Anycast Traffic from Passive Traces," *TMA*, 2016.

[6] K. E. Lindqvist and J. Abley, "Operation of Anycast Services," Internet Engineering Task Force, RFC 4786, Dec. 2006. [Online]. Available: https://www.rfc-editor.org/info/rfc4786

[7] R. Hendriks, S. Servillo, F. Cuomo, C. Hesselman, R. v. Rijswijk-Deij, and S. Kastanakis, "Too remote to be local: Latency inflation in anycast due to remote peering," in *2025 21st International Conference on Network and Service Management (CNSM)*, 2025, pp. 1–7.

[8] "What is SSL (Secure Sockets Layer)?" https://www.cloudflare.com/learning/ssl/what-is-ssl/.

[9] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/info/rfc8446

[10] E. Rescorla and T. Dierks, "The Transport Layer Security (TLS) Protocol Version 1.2," RFC 5246, Aug. 2008. [Online]. Available: https://www.rfc-editor.org/info/rfc5246

[11] D. E. E. 3rd, "Transport Layer Security (TLS) Extensions: Extension Definitions," RFC 6066, Jan. 2011. [Online]. Available: https://www.rfc-editor.org/info/rfc6066

[12] M. Lepinski and S. Kent, "Additional Diffie-Hellman Groups for Use with IETF Standards," RFC 5114, Jan. 2008. [Online]. Available: https://www.rfc-editor.org/info/rfc5114

[13] "Information Technology - Open Systems Interconnection - The Directory: Public-key and attribute certificate frameworks," International Telecommunication Union, Telecommunication Standardization Sector (ITU-T), Recommendation X.509, 2019.

[14] R.-G. Holz, "Empirical analysis of public key infrastructures and investigation of improvements," Ph.D. dissertation, Technische Universität München, 2014. [Online]. Available: https://mediatum.ub.tum.de/1182735

[15] G. Öndarö, J. Kaspereit, S. Umezulike, C. Saatjohann, F. Ising, and S. Schinzel, "S/MINE: Collecting and Analyzing S/MIME Certificates at Scale," in *USENIX*, Aug. 2025.

[16] G. L. Cesar, G. Öndarö, J. Kaspereit, F. Ising, S. Schinzel, M. Jonker, and R. Holz, "Characterizing hosting and security practices for public-facing ldap servers," in *2025 21st International Conference on Network and Service Management (CNSM)*, 2025, pp. 1–9.

[17] R. Holz, J. Hiller, J. Amann, A. Razaghpanah, T. Jost, N. Vallina-Rodriguez, and O. Hohlfeld, "Tracking the deployment of tls 1.3 on the web: a story of experimentation and centralization," vol. 50, no. 3. New York, NY, USA: Association for Computing Machinery, Jul. 2020. [Online]. Available: https://doi.org/10.1145/3411740.3411742

[18] CA/Browser Forum, "Latest Baseline Requirements," https://cabforum.org/working-groups/server/baseline-requirements/requirements/, Aug 2025.

[19] J. Nightingale, "DigiNotar Removal Follow Up," Sep. 2011.

[20] "X509-cert-insecure-signing-algorithm - Boost Security," https://docs.boostsecurity.io/rules/x509-cert-insecure-signing-algorithm.html.

[21] S. Boeyen, S. Santesson, T. Polk, R. Housley, S. Farrell, and D. Cooper, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," RFC 5280, May 2008. [Online]. Available: https://www.rfc-editor.org/info/rfc5280

[22] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and D. C. Adams, "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP," RFC 6960, Jun. 2013. [Online]. Available: https://www.rfc-editor.org/info/rfc6960

[23] A. Ashcraft, S. Cai, D. Coulter, M. Jacobs, and M. Satran, "Global Catalog - Win32 apps," https://learn.microsoft.com/en-us/windows/win32/ad/global-catalog, Aug. 2020.

[24] International Telecommunication Union, "X.500: Information technology - Open Systems Interconnection - The Directory: Overview of concepts, models and services," https://www.itu.int/rec/T-REC-X.500-201910-I/en, Dec. 2012, accessed 2023-10-11.

[25] ——, "X.501: Information technology - Open Systems Interconnection - The Directory: Models," https://www.itu.int/rec/T-REC-X.501-201910-I/en, Oct. 2019, accessed 2023-10-11.

[26] K. Zeilenga, "Lightweight Directory Access Protocol (LDAP): Directory Information Models," Internet Engineering Task Force, Request for Comments RFC 4512, Jun. 2006. [Online]. Available: https://datatracker.ietf.org/doc/rfc4512

[27] J. Sermersheim, "Lightweight Directory Access Protocol (LDAP): The Protocol," RFC 4511, Jun. 2006. [Online]. Available: https://www.rfc-editor.org/info/rfc4511

[28] R. Harrison, "Lightweight Directory Access Protocol (LDAP): Authentication Methods and Security Mechanisms," Internet Engineering Task Force, Request for Comments RFC 4513, Jun. 2006. [Online]. Available: https://datatracker.ietf.org/doc/rfc4513

[29] ——, "Lightweight Directory Access Protocol (LDAP): Authentication Methods and Security Mechanisms," RFC 4513, Jun. 2006. [Online]. Available: https://www.rfc-editor.org/info/rfc4513

[30] K. Zeilenga and A. Melnikov, "Simple Authentication and Security Layer (SASL)," Internet Engineering Task Force, Request for Comments RFC 4422, Jun. 2006. [Online]. Available: https://datatracker.ietf.org/doc/rfc4422

[31] S. Kitterman, "Rfc 7208: Sender policy framework (spf) for authorizing use of domains in email, version 1," USA, 2014.

[32] M. Kucherawy and E. Zwicky, "Domain-based Message Authentication, Reporting, and Conformance (DMARC)," RFC 7489, Mar. 2015. [Online]. Available: https://www.rfc-editor.org/info/rfc7489

[33] O. Hureau, A. Duda, and M. Korczyński, "Stress testing the dmarc reporting system: Compliance with standards and ways of improvement," in *Proceedings of the 20th International Conference on Emerging Networking EXperiments and Technologies*, ser. CoNEXT '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1–9, https://doi.org/10.1145/3680121.3697809.

[34] F. O. for Information Security, "BSI TR-03182 Email Authentication," 01 2024.

[35] G. A. Help, "About DMARC reports - Google Workspace Admin Help," https://support.google.com/a/answer/10032472?hl=en.

[36] G. L. Cesar and R. Hendrks, "TODO: a deep dive on deployment strategies of services provisioned using anycast," in *TMA*, 2026, pp. 1–11.

[37] G. L. Cesar, R. Holz, and M. Jonker, "TODO: longitudinal analysis of dmarc across country code top-level domains," in *NOMS*, 2026, pp. 1–11.

Publications by the Author

Below is a list of peer-reviewed academic publications that ... has authored or co-authored in reverse chronological order:

- usenix
- cnsm
- noms
- tma